DIGITAL TRANSFORMATION

Writing to Dataverse Tables in a Different Environment

from a Power Apps Code App

A developer reference for implementing Create, Read, Update and Delete operations across Dataverse environments — using the Semantic Model Viewer cross-environment write as the working example.

AUDIENCE

Power Apps Code App developers

STACK

Power Platform CLI (pa / pac) • TypeScript

STATUS

Verified working pattern

Contents

1 The Problem

2 The Working Solution

3 One-Time Project Setup

4 Create & Update — Writing to Another Environment

5 Read — Fetching Records from Another Environment

6 Verifying a Write Actually Landed

7 Common Mistakes

8 Extra-important Point about Pagination

TIP

How to use this guide

Each section is self-contained — jump straight to the operation you need. Sections 4–5 assume Section 3 (one-time setup) is already done for your project.

1 The Problem

A Power Apps Code App normally reads and writes Dataverse data only in the environment it is deployed to — its **home environment**. This stays true even if:

- You create a brand-new Dataverse connection while signed into a different environment.
- You pass an `--org-url` flag when adding the table as a data source.
- The table you want to write to exists, with an identical schema name, in both environments.

None of the above change where the write actually lands. The normal generated per-table service (e.g. `MyTableService.create(...)`) always writes to the app's home environment, no matter which connection ID you pass in.

> **IMPORTANT**
>
> **Don't waste time on this**
>
> Switching `pac auth` profiles, passing `--org-url` to `pa app add data-source`, or creating a fresh Dataverse connection while signed into the target environment. All of these were tested and none of them change where the write lands. Skip straight to Section 2.

2 The Working Solution

The Dataverse connector — the same connector type used by Power Automate flows — exposes a set of generic, parameterized operations that take the **target environment as an explicit argument at call time**. This is the same pattern used by the Power BI connector's `ExecuteDatasetQuery` method, which already takes a groupId and datasetId.

These operations live in **one shared, connector-level service file** (not a per-table file):

Method	Purpose
CreateRecordWithOrganization	Create a row in any environment
UpdateRecordWithOrganization	Update a row in any environment
GetItemWithOrganization	Retrieve a single row from any environment
ListRecordsWithOrganization	Query rows from any environment
DeleteRecordWithOrganization	Delete a row in any environment

TIP

No extra plumbing required

This mechanism requires no new plugin, no Custom API, and no separate authentication flow — it uses the Dataverse connection your code app already has.

NOTE

About DeleteRecordWithOrganization

Generated in the same file as the other four methods and follows the same target-environment pattern. Its exact parameter order was not exercised in the working example this guide is based on — before calling it, inspect the generated signature directly in `MicrosoftDataverseService.ts` (Ctrl+Click / go-to-definition on the method name) to confirm argument order, the same way Section 5.1 shows read methods can differ in order from write methods. Wrap the call in the same `if (!result.success)` error-handling pattern used throughout this guide.

3 One-Time Project Setup

Do this once per code app project. If your project already has the generic connector service (check step 3.3 first), skip to Section 4.

3.1 Confirm you have a Dataverse connection

```
pa connection list
```

In the output, find the row whose **Api ID** is `shared_commondataserviceforapps` — not `shared_commondataservice` (no “forapps”), which is a different connection. Copy that row's **Connection ID**; you'll pass it in Step 3.3.

Display Name	Api ID	Connection ID
you@yourcompany.com	shared_commondataservice	(ignore — wrong connector)
you@yourcompany.com	**shared_commondataserviceforapps**	← **copy this one**
you@yourcompany.com	shared_microsoftcopilotstudio	(ignore — unrelated)

3.2 Confirm you're signed in to the app's HOME environment

This matters: the connection you use here must belong to the **same environment the code app itself is deployed to** — not the environment you eventually want to write into.

```
pac auth list
```

The row marked with an asterisk (*) is active. If it isn't your code app's home environment, switch to it:

```
pac auth select --index <the-correct-index>
```

3.3 Add the connector generically (no --table flag)

This is the step that generates the shared, parameterized service.

```
pa app add data-source \
--connector shared_commondataserviceforapps \
--connection-id <your-connection-id>
```

If prompted *“Are you using a connection reference instead of a connection ID?”* — answer **No**.

Confirm the file was generated:

```
Get-ChildItem .\src\generated\services\MicrosoftDataverseService.ts
```

> NOTE
> **Do not use the --table flag**
>
> That flag generates a different, single-environment-only file instead of the shared generic service. This single file now contains every table’s generic create/update/read/delete methods for your whole app — you only add this connector once per project, regardless of how many tables or target environments you use afterward.

4 Create & Update — Writing to Another Environment

Use this pattern anywhere in your app code where you need to create or update a row that lives in a different Dataverse environment than the app itself.

4.1 Import the service

```
import { MicrosoftDataverseService } from
'./generated/services/MicrosoftDataverseService';
```

4.2 Create a record

```
const result = await MicrosoftDataverseService.CreateRecordWithOrganization(
'return=representation', // prefer — always this literal value
'application/json', // accept — always this literal value
'https://<target-org>.crm.dynamics.com', // organization: TARGET env URL
'<table_plural_logicalname>', // entityName: table's plural name
{
<field_logicalname>: '<value>', // ...more fields as needed
}
);

if (!result.success) {
throw new Error(JSON.stringify(result.error));
}
```

Parameter	Value to use
prefer	Always the literal string `return=representation`. Leave unchanged.
accept	Always the literal string `application/json`. Leave unchanged.
organization	Full URL of the TARGET environment, e.g. https://org2f45e702.crm4.dynamics.com — can be different from the app's home environment.
entityName	The table's plural logical name, e.g. cr301_accountagentmappings. Find it on the table's Columns page in the maker portal.
(final object)	A plain object of field logical name → value pairs, exactly as you'd pass to any other create() call.

4.3 Update an existing record

Same method family, with the record's GUID added as a parameter:

```
const result = await MicrosoftDataverseService.UpdateRecordWithOrganization(
'return=representation',
'application/json',
```

```
'https://<target-org>.crm.dynamics.com',
'<table_plural_logicalname>',
'<record-guid>',
{
<field_logicalname>: '<new-value>',
}
);
```

4.4 Full worked example

The exact tested example from the Semantic Model Viewer project, writing into the “Account-Agent Mapping” table (`cr301_accountagentmapping`) in a second environment.

```
import { MicrosoftDataverseService } from
'./generated/services/MicrosoftDataverseService';

async function writeToOtherEnvironment() {
const result = await MicrosoftDataverseService.CreateRecordWithOrganization(
'return=representation',
'application/json',
'https://org2f45e702.crm4.dynamics.com',
'cr301_accountagentmappings',
{
cr301_name: 'Example record',
cr301_paymenttype: 'Cash',
}
);

if (!result.success) {
throw new Error('Write failed: ' + JSON.stringify(result.error));
}

console.log('Created record:', result.data);
}
```

5 Read — Fetching Records from Another Environment

The same connector service exposes read operations that take the target environment as an explicit parameter, the same way the write methods do.

5.1 List multiple rows: ListRecordsWithOrganization

> NOTE
> **Parameter order is different here**
>
> `organization` comes **first** in this method, unlike the create/update methods above. Always check the actual method signature in MicrosoftDataverseService.ts rather than assuming the order matches.

```
const result = await MicrosoftDataverseService.ListRecordsWithOrganization(
'https://<target-org>.crm.dynamics.com', // organization
'<table_plural_logicalname>', // entityName
undefined, // prefer
'application/json', // accept
undefined, // x_ms_odata_metadata_full
undefined, // MSCRM_IncludeMipSensitivityLabel
'<field1>,<field2>', // $select — fields to return
undefined, // $filter — optional
undefined // $orderby — optional
);

if (!result.success) { throw new Error(JSON.stringify(result.error)); }
const rows = result.data;
```

Parameter	Value to use
organization	Full URL of the TARGET environment.
entityName	The table's plural logical name.
$select	Comma-separated field logical names. Always set this — an unset $select returns every column and metadata field.
$filter	Optional OData filter, e.g. cr301_paymenttype eq 'Cash'. Leave undefined to return all rows (subject to $top).
$top	Optional maximum number of rows to return.

5.2 Get a single row by ID: GetItemWithOrganization

Parameter order here matches the write methods — prefer and accept come first, then organization.

```
const result = await MicrosoftDataverseService.GetItemWithOrganization(
'return=representation', // prefer
'application/json', // accept
```

```
'https://<target-org>.crm.dynamics.com', // organization
'<table_plural_logicalname>', // entityName
'<record-guid>', // recordId
undefined, // x_ms_odata_metadata_full
undefined, // MSCRM_IncludeMipSensitivityLabel
'<field1>,<field2>' // $select
);

if (!result.success) { throw new Error(JSON.stringify(result.error)); }
const row = result.data;
```

TIP

Same setup, same file

Both read methods live in the same MicrosoftDataverseService.ts generated in Section 3.3 — there is no separate setup step for reading versus writing. If Section 3 is done for writes, reads are already available with no further configuration.

6 Verifying a Write Actually Landed

Do not trust the on-screen success message alone. Verify the record physically:

1. Open the target environment directly in a browser.
2. Switch to that environment in the top-right environment picker if using make.powerapps.com.
3. Go to **Tables** → **(your table)** → **Data**.
4. Confirm the new/updated row is present with the expected values.

IMPORTANT

Why this step matters

A write can appear to succeed in the app while actually having landed in the wrong environment — this exact mistake happened several times during development of this pattern before the correct method was found.

7 Common Mistakes

IMPORTANT

Using the per-table generated service instead

Running `pa app add data-source --table <name>` gives you a dedicated class like `MyTableService.create(...)`. This class has **no way** to specify a target environment — it always writes to the app's home environment. Use MicrosoftDataverseService (Section 3.3) instead for any cross-environment write.

IMPORTANT

Wrong environment URL format

The `organization` parameter expects the full `https://` URL of the target environment, exactly as it appears in the browser address bar — not just the org name or the environment ID GUID.

IMPORTANT

Missing security role in the target environment

The signed-in user still needs a Dataverse security role with Create/Write privileges on the target table **inside the TARGET environment**. Licensing and connection setup do not substitute for this — check Users + permissions → Security roles in the target environment's Power Platform admin center settings if writes fail with a permissions error.

8 Extra-important Point about Pagination

Dataverse caps every response at **5,000 rows per page**, regardless of `$top`. If the table has more rows than that, `@odata.nextLink` will be present in the response and must be followed in a loop.

IMPORTANT

Do not use $top if you want all the rows

Setting `$top` tells Dataverse "this is all I want," and it will **not** return `@odata.nextLink` even if more rows exist. Instead, control page size through the `prefer` parameter:

```
prefer: 'odata.include-annotations="OData.Community.Display.V1.FormattedValue",odata.
maxpagesize=5000'
```

The nested nextLink problem

This connector's `@odata.nextLink` is not a flat Dataverse URL. It points at the connector's own `nextLink` invoke endpoint, with the real continuation query buried — URL-encoded — inside a `next` parameter. A naive extraction will always return `null` and silently stop the loop after one page, with no error.

Code before — breaks silently after page 1

```
const nextLink = data['@odata.nextLink'];
const qs = nextLink?.split('?')[1];
const skiptoken = qs ? new URLSearchParams(qs).get('$skiptoken') : undefined;
```

Code after — correctly unwraps the nested query

```
const nextLink = data['@odata.nextLink'];
let skiptoken: string | undefined;

if (nextLink) {
const outerUrl = new URL(nextLink);
const innerQueryString = outerUrl.searchParams.get('next');
const questionIdx = innerQueryString?.indexOf('?') ?? -1;

if (innerQueryString && questionIdx !== -1) {
const innerParams = new URLSearchParams(innerQueryString.substring(questionIdx + 1));
skiptoken = innerParams.get('$skiptoken') ?? undefined;
}
}
```

TIP

How to use it

Loop `ListRecordsWithOrganization`, passing `skiptoken` each time, until no `@odata.nextLink` is returned.